PullUp Architecture

Oct 24, 2026 · @Peter Liu

Overview

PullUp gives a friend group (a "crew") one AI-picked side quest at a real nearby place each week, checks that they did it with a photo, and turns the month into a shareable comic. There is no account and no download: the crew joins through a link pasted into the group chat they already use.

Demo story (about 3 minutes):

1. A judge taps the invite link, types a name, picks a few vibes, and is in the crew in about 30 seconds.
2. Someone taps **Pull up**. The agent proposes 3 quests built from real places and events near campus, and the crew votes.
3. A teammate uploads a photo. The AI confirms the quest was done and writes a funny caption.
4. Show the monthly recap comic (three weeks pre-seeded) and share it to an Instagram Story.
5. Stretch: ask Muse in WhatsApp "what's our PullUp this week?" and it answers from our connector.

Which part earns which prize:

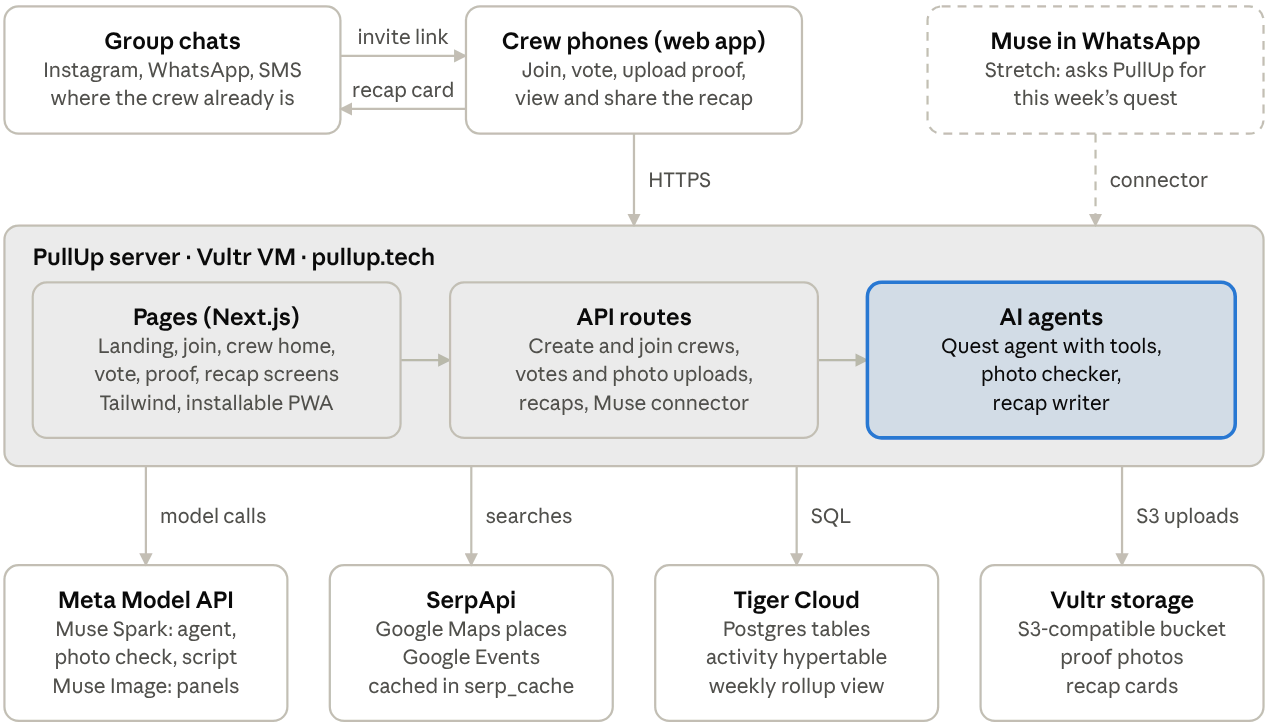
Track	Kind	What earns it in PullUp
Meta: Bringing People Closer Together with AI	Corporate, 3 winners	Muse Spark runs the quest agent and the photo check; Muse Image draws the recap; Muse connector as stretch
Best Use of SerpApi	Corporate, 1 winner	Google Maps and Google Events results are the raw material for every quest
Most Entertaining (Jester's Court)	HackTX, 1 winner	Voting, photo proof, AI captions, comic recap

Track	Kind	What earns it in PullUp
Best Use of Tiger Data	MLH	All app data, plus a time-series activity log with a continuous aggregate for crew stats
Best Use of Vultr	MLH	App server and object storage for photos
Best .Tech Domain	MLH	pullup.tech (confirm it is free on get.tech first)
Best Use of Gemini API	MLH, optional	Enter only if Gemini ends up doing real work, such as the recap images if Muse Image is not available
Best Overall, Best Novice, Best Pitch	Automatic	The whole project

System architecture

Phones only ever talk to the PullUp server; the server's AI agents call Muse and SerpApi, and everything is saved in Tiger Cloud and Vultr storage.

One Next.js server on Vultr sits between the crew and four services



PullUp architecture · phones, one server, four services

The dashed parts are the stretch goal. Because the phones never call outside services directly, API keys stay on the server and a fallback model or cached data can be swapped in without touching the app.

Tech stack

One Next.js app serves both the pages and the API, so there is one thing to build, one thing to deploy, and a framework agentic coding tools know well.

Layer	Choice	Why
Web app	Next.js (App Router, TypeScript) + Tailwind, installable as a PWA	Pages and API routes in one codebase; runs in any phone browser, no app store
AI model	Muse Spark 1.1 on Meta Model API, called with the OpenAI SDK pointed at `api.meta.ai/v1`	Meta track; handles tool calls, structured JSON output and photo input

Layer	Choice	Why
Recap images	Muse Image if it is in the API this weekend, otherwise Gemini image generation	Draws the comic panels
Place data	SerpApi `google_maps` and `google_events` engines	Real places and events with addresses, ratings, hours and coordinates
Database	Tiger Cloud (Postgres + TimescaleDB) through Drizzle ORM	Normal tables plus a time-series activity log
Photo storage	Vultr Object Storage (S3-compatible) through the AWS S3 SDK	Proof photos and recap images
Hosting	Vultr cloud compute VM running Node with pm2, Caddy in front for HTTPS	Caddy gets the certificate for pullup.tech automatically
Image compositing	`sharp`	Stitches panels and stats into a 1080×1920 story card
Sessions	Signed cookie (JWT via `jose`) issued when someone joins	No passwords, no sign-up form

All model calls go through one small client file whose base URL, key and model name come from environment variables. Switching between Muse and a fallback model is a config change, not a code change.

Data model

Eight ordinary tables hold the app state, and one hypertable logs every action so recaps and crew stats come from a single query.

Table	Key columns	Notes
`users`	id, name, avatar_url, interests (text[]), created_at	Created on join; interests are the vibe tags picked at join

Table	Key columns	Notes
`crews`	id, name, invite_code, home_label, home_lat, home_lng, radius_km, created_by	Home area is set once by the creator
`crew_members`	crew_id, user_id, joined_at	Primary key on (crew_id, user_id)
`quests`	id, crew_id, week_start, status, title, task, place_name, address, lat, lng, source, source_ref, proof_hint, est_cost	Status is proposed, active, done or expired; 3 proposed per week, the vote winner becomes active
`quest_votes`	quest_id, user_id	One vote per person per week
`proofs`	id, quest_id, user_id, photo_key, verdict, confidence, caption, created_at	Verdict is pass, unsure or fail
`recaps`	id, crew_id, period_start, period_end, script (jsonb), image_key	One per crew per month
`serp_cache`	query_hash, engine, response (jsonb), fetched_at	Saves SerpApi credits and lets the demo replay results offline
`activity` (hypertable)	time, crew_id, user_id, kind, quest_id, meta (jsonb)	Kinds: joined, voted, proof_submitted, quest_completed, recap_made

The hypertable and its weekly rollup are plain SQL in a migration file:

```
CREATE TABLE activity (
  time     timestamptz NOT NULL DEFAULT now(),
  crew_id  uuid NOT NULL,
  user_id  uuid,
  kind     text NOT NULL,
  quest_id uuid,
  meta     jsonb DEFAULT '{}'
);
SELECT create_hypertable('activity', 'time');

CREATE MATERIALIZED VIEW crew_weekly
WITH (timescaledb.continuous) AS
```

```
SELECT time_bucket('7 days', time) AS week,
       crew_id,
       count(*) FILTER (WHERE kind = 'quest_completed') AS quests_done,
       count(*) FILTER (WHERE kind = 'proof_submitted') AS photos,
       count(*) FILTER (WHERE kind = 'voted')           AS votes
FROM activity
GROUP BY week, crew_id;

SELECT add_continuous_aggregate_policy('crew_weekly',
  start_offset => INTERVAL '2 months',
  end_offset   => INTERVAL '1 hour',
  schedule_interval => INTERVAL '15 minutes');
```

The crew home page and the recap card read their streak and totals from `crew_weekly`. That is the Tiger Data story for judging: relational state and time-series history in one Postgres database.

AI layer

Three AI jobs share one model client: a quest agent that uses tools, a photo checker, and a recap writer. Only the quest agent is a true agent loop; the other two are single calls with a fixed JSON output.

Quest agent (`lib/agents/questAgent.ts`)

Runs when someone taps **Pull up**. Muse Spark gets the crew profile and three tools, calls them as it sees fit (capped at 6 rounds), then returns exactly 3 quests.

What it receives: each member's interest tags, crew size, home area and radius, day of the week, and the last 8 quests so it does not repeat itself.

Tools it can call:

Tool	Backed by	Returns (trimmed to keep prompts small)
`search_places(query, near)`	SerpApi `google_maps`	Top 8: name, address, type, rating, hours, coordinates, place id
`search_events(query, near)`	SerpApi `google_events`	Top 8: title, date, venue, address, link

Tool	Backed by	Returns (trimmed to keep prompts small)
`get_past_quests()`	`quests` table	Last 8 titles and places

Output (structured JSON):

```
{
  "quests": [
    {
      "title": "Vinyl Roulette",
      "hook": "One line that sells it to the group chat",
      "task": "What the crew actually does there",
      "place": { "name": "", "address": "", "lat": 0, "lng": 0, "source_ref":
"" },
      "proof_hint": "What the photo should show",
      "why_this_crew": "Which shared interests it hits",
      "est_cost": "free | $ | $$",
      "time_needed_min": 90
    }
  ]
}
```

Rules in the system prompt: public, legal, safe places only; doable in 2 to 3 hours; at least one free quest; spread across interests so everyone gets something they like; only use places returned by a tool.

Guard in code: after the model answers, check that every `source_ref` appears in that run's tool results. Drop any quest that fails and ask for a replacement. This stops invented places from ever reaching the screen.

Photo checker (`lib/agents/verifyProof.ts`)

One call with the photo (resized to about 1024 px) plus the quest's task and proof hint. It returns `{ verdict: pass | unsure | fail, confidence, reason, caption }`, where the caption is a playful line of 20 words or fewer. An **unsure** result goes to the crew, who can approve it with a tap, so a wrong AI call never blocks the fun.

Recap writer (`lib/agents/recap.ts`)

1. Muse Spark reads the month's completed quests and captions and writes a 4-panel comic script as JSON (scene and one line of dialogue per panel).

2. The image model draws each panel with one fixed style prompt so the panels match.
3. `sharp` lays the panels out in a 2×2 grid on a 1080×1920 card with the crew name and stats from `crew_weekly`.

Generate the demo crew's recap ahead of time, then show live generation only if there is time.

Shared client (`lib/ai.ts`)

```
import OpenAI from "openai";

export const ai = new OpenAI({
  baseURL: process.env.AI_BASE_URL, // https://api.meta.ai/v1
  apiKey: process.env.AI_API_KEY,
});
export const MODEL = process.env.AI_MODEL; // muse-spark-1.1
```

Follow Meta's quickstart for which request style to use (it shows the Responses API). Keep every prompt in its own file so the AI owner can tune prompts without touching app code.

Core flows

Four flows make up the whole product, and each one writes a row to `activity` when it finishes.

A. Create and join a crew

1. The creator enters a crew name and home area (a neighborhood name, or the phone's location if allowed). The server saves the crew and an invite code.
2. The creator taps **Share invite**; the phone's share sheet sends `pullup.tech/j/CODE` into any group chat.
3. A friend opens the link, types a name, and taps 4 to 5 vibe chips (food, music, outdoors, games, art, sports, cheap eats, and so on).
4. The server creates the user, adds them to the crew, and sets a signed session cookie. They land on the crew home page.

B. Get this week's quest

1. Anyone taps **Pull up**. The server checks there is no active quest this week.
2. The quest agent runs (see AI layer), and 3 proposed quests are saved.

3. Members vote with one tap each. When most of the crew has voted, or after 24 hours, the top quest becomes active and the others expire.
4. The crew home page shows the active quest with a map pin, address, cost and proof hint.

C. Prove it

1. Someone taps **We pulled up**, takes or picks a photo, and sends it.
2. The server stores it in object storage and runs the photo checker.
3. Pass: the quest is marked done and the caption appears in the crew feed. Unsure: the crew sees an **Approve** button. Fail: a friendly retry message.

D. Recap and share

1. At month end (or by tapping **Make recap** in the demo), the recap writer builds the comic card.
2. The card shows in the app with a **Share** button. On phones, the share sheet sends the image straight to Instagram Stories, WhatsApp or anywhere else, with no Meta API needed.

Muse connector (stretch goal)

The connector lets people reach PullUp from Muse in WhatsApp; start it only after the core demo works end to end, around hour 16. Muse accepts custom connectors where the developer supplies the API and Muse handles the conversation, but the exact format and access rules for this weekend are unconfirmed.

- [ ] Hour 0: ask the Meta booth what format a custom connector takes (OpenAPI spec, manifest, auth method) and whether hackers can add one to a test account this weekend

Planned actions (thin wrappers over the same service functions the web app uses, so this is about 2 hours of work):

Action	What it does
`list_my_crews`	Crews the user belongs to
`get_current_quest(crew)`	This week's active quest, or the 3 proposals if voting is open
`vote(crew, quest)`	Casts the user's vote

Action	What it does
`start_quest_generation(crew)`	Same as tapping Pull up
`get_recap_link(crew)`	Link to the latest recap card

Auth: a per-user connector token shown on a **Connect to Muse** settings screen, unless Meta requires OAuth, in which case follow their spec.

Demo line: "PullUp also lives inside the AI agent your friends already text." If the connector cannot be set up in time, show the endpoints and spec in the pitch as the next step rather than faking it.

Repo, routes and environment

Keep pages, API routes and the shared `lib/` code separate so four people can work in parallel without merge fights.

```
pullup/
  app/
    page.tsx                      landing + create crew
    j/[code]/page.tsx             join from invite link
    c/[crewId]/page.tsx           crew home: quest, votes, feed
    c/[crewId]/recap/page.tsx     recap card + share
    api/
      crews/route.ts              POST create crew
      crews/[id]/join/route.ts    POST join
      crews/[id]/generate/route.ts POST run quest agent
      quests/[id]/vote/route.ts   POST vote
      quests/[id]/proof/route.ts  POST photo (multipart) -> store -> verify
      proofs/[id]/approve/route.ts POST crew approves an unsure photo
      crews/[id]/recap/route.ts   POST build recap, GET latest
      connector/[action]/route.ts Muse connector (stretch)
  lib/
    ai.ts           shared model client
    serp.ts         SerpApi wrapper + cache
    storage.ts      Vultr Object Storage upload/download
    session.ts      signed cookie helpers
    db/schema.ts    Drizzle tables
    db/index.ts     connection
    agents/questAgent.ts
    agents/verifyProof.ts
```

```
    agents/recap.ts
    prompts/        one file per prompt
  drizzle/          migrations, including the hypertable SQL
  scripts/seed-demo.ts   demo crew with 3 weeks of history
```

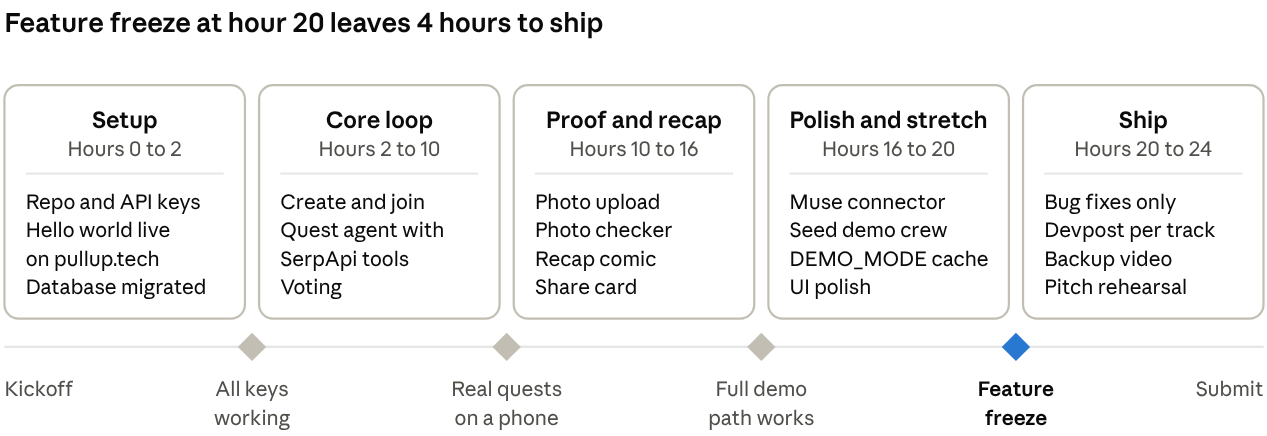
Photos go to the server as a normal form upload and the server pushes them to storage. That avoids browser upload permission (CORS) setup, which is not worth the time this weekend.

Variable	Used for
`AI_BASE_URL`, `AI_API_KEY`, `AI_MODEL`	Muse Spark (or a fallback model)
`IMAGE_PROVIDER`, `IMAGE_API_KEY`	Recap panel images
`SERPAPI_KEY`	Places and events
`DATABASE_URL`	Tiger Cloud connection string
`S3_ENDPOINT`, `S3_BUCKET`, `S3_ACCESS_KEY`, `S3_SECRET_KEY`	Vultr Object Storage
`SESSION_SECRET`	Signing session cookies
`DEMO_MODE`	Replays cached AI and SerpApi answers if the network fails

Put the variable names (with empty values) in a committed `.env.example` and share the real keys through a private team note, never the repo.

Build plan and team split

The plan has five phases, each ending at a check the whole team confirms before moving on; feature freeze is at hour 20 no matter what.



24-hour build roadmap · 5 phases, 4 checks

Each diamond is a check the team confirms together; if a check is missed, cut scope from the next phase rather than pushing the freeze.

Role	Owns	Phase 1 job (hours 0 to 2)
Frontend	All pages, mobile layout, vote and proof screens, recap share	Scaffold Next.js + Tailwind, landing and join pages
AI	Quest agent, photo checker, recap writer, prompts	Get a Muse Spark key working; first tool call to SerpApi
Data	Tiger schema and migrations, SerpApi wrapper and cache, storage, seed script	Tiger Cloud database, Drizzle schema, hypertable migration
Infra and pitch	Vultr server, domain, deploys, Muse connector, Devpost, slides, demo video	Register pullup.tech, Vultr VM with Caddy, deploy hello world

If your team has three people, merge Data into Infra and pitch.

Risks and fallbacks

The biggest risk is API access, not code, so every outside service has a fallback that is a config switch.

Risk	Fallback
Meta Model API key is waitlisted or slow	Ask the Meta booth at hour 0. Meanwhile build against another OpenAI-compatible model (Gemini offers one) by changing the three `AI_` variables; switch back to Muse before judging, since the Meta track needs it
Muse Image is not in the API	Draw recap panels with Gemini and say so plainly in the Devpost write-up
SerpApi credits run low or calls are slow	`serp_cache` stores every response; cap each quest run at 3 searches; the demo crew uses cached results
The agent invents a place	The code guard drops any quest whose place is not in that run's tool results
Venue Wi-Fi fails during judging	`DEMO_MODE` replays cached answers; record a 2-minute backup video by hour 22
Server setup eats hours	Deploy hello world in hour 1. If Vultr is still not working at hour 3, host elsewhere temporarily and move back by hour 16
Photo check gets the demo photo wrong	Test the exact demo photos ahead of time; the Unsure path lets the crew approve
Privacy and safety questions from judges	Public places only, no live location sharing, photos visible only to the crew